

A Hierarchical BRAM/URAM Buffer for SNN

Rasha Karakchi

Dept. Computer Science and Engineering
University of South Carolina
Columbia, SC
karakchi@cec.sc.edu

Abstract—Applications such as facial recognition, social networking and automated translation are well-known by their exponential growth in size and level of complexity. Such applications are the real-world examples of machine learning which are computationally modeled using neural network. Executing such operations are bounded by the needed memory to store and transfer datasets into computational units. Access rate and storage capacity are related, in somehow, to the type of memory and its structure. In domain-specific architectures, on_chip memories are usually utilized as data memory because they allow wider busses and higher operation rate, however they are less dense than off_chip memories such as DRAMs. Examples of on_chip memories are Block RAM (BRAMs) and UltraRAMs (URAMs). While both memories are equivalent in latency and speed, URAM is considered denser because of its capability to cascade and create larger memory without reversely impacting the frequency. In this work, we propose a hierarchical hybrid input buffer to store weight and spikes of spiking neural network accelerator. The buffer is designed as a combination of BRAMs and URAMs where BRAMs represent the lower layer (closer to processing unit) while URAM are the upper layer. We tested our design on Xilinx Ultrascale Zynq 104 FPGA device and we found that our design can run on a higher frequency with less hardware resources and BRAM/URAM utilization comparing with the state-of-the-art.

Index Terms—Ultra RAM (URAM), Spiking Neural Network, FPGA, BlockRAM, Domain-Specific Architectures

I. INTRODUCTION

Facial recognition, automated translation and social networking applications are data-intensive applications that handle large quantity of complex data which are distributed across a number of units [1]–[4]. In recent years, algorithms such as conventional neural networks (CNNs), spiking neural networks (SNNs), finite automata have been exploited to serve as the computational models for such data because of their capability to process data in multi_step pipelines [5]–[7]. One of the main bottlenecks of executing data-intensive applications is the massive amount of data that need to be transferred between the buffers and the processing units that implemented as a hardware pipeline which consists of a number of processing elements (PEs) that can be programmed to execute same tasks on a different data sets, known as systolic arrays. For example, the operation of spiking neural network requires executing a

read operation from the input buffer to transfer weight and spikes to the processing units at every clock cycles.

In general, for large datasets, computer systems are limited by memory capacity and bandwidth. Nowadays, we are in the era of big data where datasets are growing exponentially, this results in more memory capacity needed with higher transfer rate. To achieve high throughput and low latency, a shorter gap between the processor and memory has become more crucial. Several factors determine the memory capacity and latency including the memory fabrication such as static RAM (SRAM) versus Dynamic RAM (DRAM), memory structure such as sparse versus dense memory, and the distance between memory and computational units. Efficiency and scalability are significantly affected by memory capacity and latency. Therefore, in recent years, research groups have been exploring different strategies to tackle the long latency and the not-enough memory capacity in order to improve overall performance [8]–[12].

Memory hierarchy is a core technique to mitigate the memory bound in Von-Neumann architecture. Hierarchical memory is implemented as a set of layers where faster memory (cache) is closer to processor where slower memory (DRAM) is more distant. It's a trade-off, faster memory is more expensive and smaller. While general-purpose computers are designed to execute any mathematical operation, the continuous growth in data size and complexity have made such computers unsuitable for all type of operations, specially for data-intensive applications. This led to the evolution of Domain-Specific Architectures (DSAs).

Example of DSAs are Field Programmable Gate Arrays (FPGAs) and Application-Specific Integrated Circuits (ASICs) which have the capability to be programmed to execute specific tasks. While ASIC can be programmed only for one-time, FPGAs have the ability to be re-programmed. FPGAs are manufactured as a System on Chip (SoC) where one chip is a complete system that consists of processor units, controllers and on_chip memory, as shown in Figure 1. On_chip memory, represented by Block RAMs (BRAMs) and Ultra RAMs (URAMs) in Figure 1, are fast because of they are close to processor and no I/O overhead that slows overall execution time (unlike off_chip DRAM memory). Because of its low

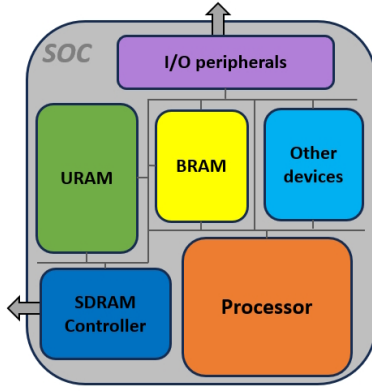


Fig. 1. FPGA as System on Chip

latency, BRAMs are frequently implemented as buffers to store instructions [11], or data [8]–[10], [13], however they are limited in capacity. URAMs, on the other hand, can have higher depth than BRAMs but with limited width. Each URAM block has a capacity 8x as large as a BRAM block. While both BRAMs and URAMs have same latency of the operations (one clock for read, one for write), URAM has a deep internal pipeline for data readout and thus, it is capable of cascading multiple URAM blocks without sacrificing its maximum clock frequency f_{max} [9]. Whereas BRAM's f_{max} drops when blocks are cascaded.

While the majority of previous work in spiking neural network on FPGAs used the BRAMs for buffering [10], [14]–[17], few groups utilized both URAMs and BRAMs for buffering. Aung et al. [9] designed a cascaded combination of URAM and BRAM to allocate more memory to cover more neurons. BRAM blocks are cascaded, however, they result in dropping its maximum frequency (f_{max}) [9]. Cascading the BRAM and URAM modules leads to forming larger memory but it affects the performance. Therefore, in DeepFire2 model the usage of URAM is preferred for wider layers. For larger models, up to 16 BRAM blocks had to be cascaded due to a lack of URAM resources [9].

In this paper, we present a hierarchical hybrid combination of BRAM and URAM buffering for spiking neural network accelerator. BRAM blocks are utilized as the lower layer of memory that sends weight and spikes into the processing units while URAMs are designed as the upper layer in the hierarchy which serves as the feeder (source) for BRAMs. As an output buffer, BRAMs are designed similar to the weight input buffer but used as a transformer that receives data and sends it out to URAMs. In this work, we explore the impact of using small blocks of BRAMs comparing to large blocks through partitioning (or dividing) the BRAMs buffer that stores the spikes into a smaller blocks. This work is comprised of two main contributions:

- 1) Implementation of spiking neural network accelerator which comprises of an array of hardware LIF-based neurons, and the BRAMs and URAMs hierarchical buffering.

- 2) Analysis of the tradeoffs between network capacity, hardware resources, power consumption and on-chip memory and comparisons to state-of-the-art FPGA-based SNN designs.

II. RELATED WORK

On-chip memories have been used as input/output storage for many computational models such as conventional neural networks [18]–[29], spiking neural networks [8], [10], [15], [17], [30]–[36] and pattern matching [6], [11], [37].

Zhao et. al [26] designed the convolutional neural network on FPGAs and reduced amount of on-chip memory resources by quantizing the weight and activation into binary values to reduce the memory consumption and speed up the computation. On-chip memory allows FPGAs to be reconfigured with new dataset such as [11] where BRAMs are designed to store large datasets for pattern matching accelerators. SyncNN [38] proposed a novel synchronous event-driven SNN reconfigurable inference engine and evaluated multiple SNN models on multiple FPGAs. Firefly [17] is an RTL-based spiking neural network accelerator that focused on optimizing spatial parallelism on FPGA through optimizing the Multiply and Adder (MAC) unit by incorporating DSP optimization techniques into SNN synaptic operations, as well as using on-chip memory design to store the weight and improve memory efficiency. The buffer of DeepFire2 [9] is designed based on cascading BRAM blocks and URAMs clock to achieve larger memory. While this design did not cause any sacrifice in the URAM_ f_{max} , the throughput lowered because of the drop in BRAM_ f_{max} .

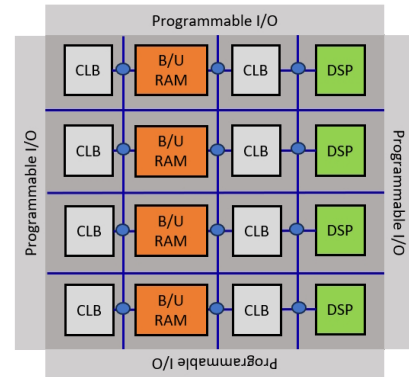


Fig. 2. FPGA Substrate

III. BACKGROUND

A. FPGAs and Onchip memories

Field Programmable Gate Arrays (FPGAs) are programmable silicon chips that are commonly designed for specific purposes, known as Domain Specific Architectures (DSAs) [39]. Modern FPGAs consist of millions of logical cells, known as configuration logical blocks (or CLBs) that can be configured to implement a variety of functions. The CLB basic is a look-up table (LUTs) and registers (or D-Flip Flops).

As shown in Figure 2, CLBs communicates with each other and with other components through local interconnections (blue lines). Switches (blue bubbles) are used for wire sharing. Block RAMs, Ultra RAMs and DSPs are distributed in FPGA as a set of components that are connected to other components through the local interconnections. Programmable I/O allows the on_chip components to communicate with off_chip devices.

B. Spiking neural networks

Spiking Neural Networks (SNNs) are the third generation of neural networks whose behavior is similar to the biological human brain [10], [13], [16]. Spiking neural network operates on *spikes* happening at discrete events which is different from conventional neural networks that operate at continuous values. Spikes are binary values which are driven from predecessors. The first input spikes are usually stored in an input buffer, similar to the weight that are stored in a separate buffer. If the neuron is active, spikes are fired to its successors. Neurons' activity depends on the incoming spikes and the current membrane level which has to be lower or equal to a specific threshold to fire a spike [10], [40].

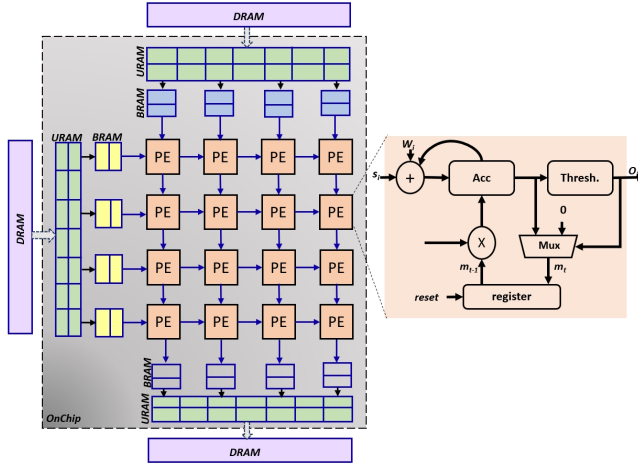


Fig. 3. SNN array including the neuron design and the hierarchical buffer

IV. ARCHITECTURES

We designed our spiking neural network accelerator as a two-dimensional array of processor elements (PEs) where each processor element represents a neuron. Our neuron is adapted from S2N2 [10]. Figure 3 shows the implementation of our accelerator. Each processor element (PE) reads the weight from an input buffer implemented using a BRAM while the input spikes are implemented using a separate BRAM. The output buffer is implemented similar to the weights'. Figure 4 shows the buffer hierarchy of the BRAM and URAM. BRAMs are distributed as a set of horizontal blocks (colored in blue) and vertical blocks (colored in yellow). The width of the horizontal BRAMs BW can be either 8-bit, 16-bit, 32-bit or 64-bit based on the weight width (WW), while the depth BD varies depending on the available resources. The

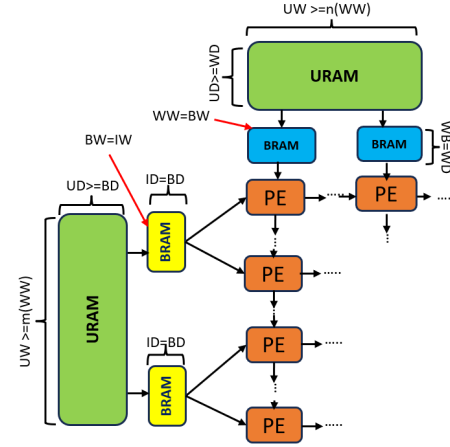


Fig. 4. The Hierarchical Memory of SNN Accelerator

vertical BRAMs are partitioned based on the number of rows m that are covered in BRAM. The width of each vertical BRAMs BW equals to number of spikes which is related to the number of rows that are included in each BRAM. For example, if five PEs (or rows) in each BRAM, that means $BW = 5$, given the fact that spikes are binary values. If the number of rows in an array is 100, 20 vertical BRAM blocks are implemented. The structure of the BRAM (width and depth) can vary based on the hardware resources. The green blocks in Figure 4 represents the larger memory that provides new data to the BRAMs anytime all locations of the BRAMs are read (or used). Sending a new batch of data into BRAMs are done using two counters. First counter increments by one until all locations of BRAMs (vertical and horizontal) are read. Once the counter values reaches the BRAM depth, BRAM values are reset and the second counter increments by one to allow streaming a new batch of data from the URAM into the BRAMs. The width of the horizontal URAM is equal to $UW = n * WW$, where n is the number of horizontal BRAMs and WW is equal to the weight width. The width of the vertical URAM equals to $UW = m * IW$, where m is the number of horizontal BRAMs and IW is equal to the input width.

V. EVALUATION AND RESULTS

Our design is mapped onto commercially available Xilinx Zync104 Ultrascale FPGA. Our proposed design is implemented using VerilogHDL. The verilog codes are synthesized and implemented in the Xilinx Vivado 2021.2 with ML-based design optimization to achieve a higher clock rate. We evaluated our design on UltraScale Zync104 FPGA device which consists of 225K LUTs, 450K registers, memory, 312 BRAMs and 96 URAMs [41]. We gathered our results based on the following three test cases:

A. Hardware Resources Versus Spike BRAM Sets

Previous work [8], [9], [17] designed the input buffer as one BRAM block that stores spikes. Spikes are binary values so each bit in the BRAM represents a spike. Multiple spikes

are streamed into the input neurons at one time. In Xilinx Zync104 FPGA device, the bigger buffer size that can be implemented is 256x32K, where 256 is the width and 32K is the depth. This means that 256 spikes can be streamed at one time. To allow for more spikes to stream at one clock cycle, we implemented a set of BRAMs, however at the cost of more memory utilization and power consumption as shown in Figure 5. The x-axis represents the number of BRAMs

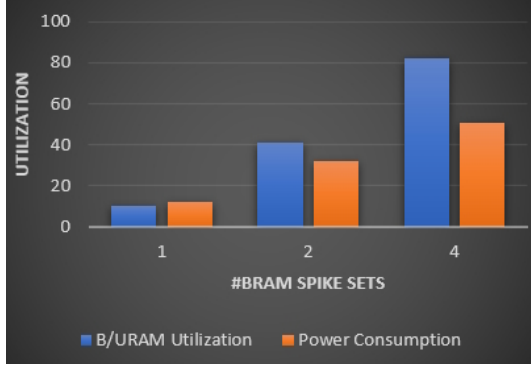


Fig. 5. Hardware Resources Utilization of SNN Accelerator with Different Input Sets

sets that are selected to store spikes and stream them into the array while the y-axis represents the power consumption and on_chip memory utilization. Since spikes that enter the input neurons are binary (1-bit value), the width bits equals the number of input neurons that receive spikes from the BRAM. The measurement assumes that BRAM_width is 256-bit and depth is 32K. Due to device hardware limitation, we were able to implement only 4 BRAM sets. As shown in Figure 5, the B/URAM utilization and power consumption increase while LUTs utilization is constant to 34%.

B. Fmax versus Power Consumption

Fmax is the maximum clock frequency that digital circuit operates. In systolic array, number of operations executed per second is in proportional relationship with Fmax. If Fmax is higher, peak GOP (Giga operations) per second is higher, however at the cost of more power consumption as shown in Table I. We ran our design on a range of Fmax (200-500 MHz) for an array size of 144x16. As shown in the table, the peak GOP per second is the highest at 500MHz while power consumption is the lowest at 200MHz. The hardware utilization is constant with the frequency change.

TABLE I
HARDWARE RESOURCES UTILIZATION WITH DIFFERENT FMAX VALUES

Fmax (MHz)	LUT Utilization	Power (Watt)	Peak GOP
500	34%	1.721	2250
400	34%	1.718	1800
300	34%	1.716	1350
200	34%	1.712	900

C. Our proposed design versus DeepFire2 state-of-the-art

We compared our design with DeepFire2 [9] which has an input buffer designed as a unified cascaded units of BRAM and URAM. DeepFire2 targeted VCU118 FPGA platform which is much bigger device than ours as it contains 1.12 MLUTs, 2160 BRAMs and 960 URAMs. Our results shown in Table 4 are for an array of 144x16, where the weight_width is 8, number of BRAMs for weight and output buffers are 16 and 64kB depth, in addition to two URAM Blocks. We compared DeepFire2 with two models. The first row in the table represents the first model which consists of one spike BRAM buffer of 144 bit width and 64KB depth and one URAM block. The second row represents the second model that consists of four spikes buffers (sets), where each consists of 36 bits width and 64KB depth and one URAM block. The DeepFire2 array size is 62 times bigger than our array. Depending on theoretical observation, If we run 144x16 array on DeepFire2 model, this roughly requires 50% memory utilization and 46% LUTs utilization which is much higher resources than our design required.

VI. CONCLUSION

In this paper, we proposed the hierarchical hybrid on_chip memory for spiking neural network buffering. We explored the benefits of partitioning the buffer into BRAM sets in order to embrace larger size of dataset, specifically spikes. Our results showed that more partitions leads to higher memory utilization and power consumption but LUTs utilization remains constant. We could run our design on 500MHz Fmax. Higher Fmax improves the throughput but at the cost of more power consumption. For future work, we plan to run our design on largest FPGA platforms to make a fair comparison with state-of-the-art. We also plan to test our design on some of the available benchmarks such as MNIST, CiFar-10 and ImageNet.

REFERENCES

- [1] A. Elmahmudi and H. Ugail, "Deep face recognition using imperfect facial data," *Future Generation Computer Systems*, vol. 99, pp. 213–225, 2019.
- [2] G. Zhao, H. Yang, and M. Yu, "Expression recognition method based on a lightweight convolutional neural network," *IEEE Access*, vol. 8, pp. 38 528–38 537, 2020.
- [3] Y. Cai, J. Llorca, A. M. Tulino, and A. F. Molisch, "Compute-and data-intensive networks: The key to the metaverse," in *2022 1st International Conference on 6G Networking (6GNet)*. IEEE, 2022, pp. 1–8.
- [4] A. Sapountzi and K. E. Psannis, "Social networking data analysis tools & challenges," *Future Generation Computer Systems*, vol. 86, pp. 893–913, 2018.
- [5] M. F. Nuryasin, C. Machbub, and L. Yulianti, "Abnormal event behavior detection for home surveillance cameras system," in *2023 IEEE 13th International Conference on System Engineering and Technology (ICSET)*. IEEE, 2023, pp. 7–11.
- [6] R. Karakchi, L. O. Richards, and J. D. Bakos, "A dynamically reconfigurable automata processor overlay," in *2017 International Conference on Reconfigurable Computing and FPGAs (ReConFig)*. IEEE, 2017, pp. 1–8.
- [7] A. Pradhan, J. King, S. Pinisetty, and P. S. Roop, "Model based verification of spiking neural networks in cyber physical systems," *IEEE Transactions on Computers*, 2023.
- [8] M. T. L. Aung, C. Qu, L. Yang, T. Luo, R. S. M. Goh, and W.-F. Wong, "Deepfire: Acceleration of convolutional spiking neural network on modern field programmable gate arrays," in *2021 31st International Conference on Field-Programmable Logic and Applications (FPL)*. IEEE, 2021, pp. 28–32.

TABLE II
PROPOSED SNN ACCELERATOR COMPARED WITH DEEPFIRE2 [9].

	Device	Array size	Fmax (MHz)	LUTs Utilization	B/URAM Utilization	Power (Watt)	Peak GOP
One spike BRAM Set	144x16	Zync104	500	34%	36.685%	1.721	2250
Four spike BRAM Sets	144x16	Zync104	500	34%	24.905%	1.721	2250
DeepFire2	140K neurons	VCU118	450	75.00%	81.34/100%	–	543

- [9] M. T. L. Aung, D. Gerlinghoff, C. Qu, L. Yang, T. Huang, R. S. M. Goh, T. Luo, and W.-F. Wong, "Deepfire2: A convolutional spiking neural network accelerator on fpgas," *IEEE Transactions on Computers*, no. 99, pp. 1–11, 2023.
- [10] A. Khodamoradi, K. Denolf, and R. Kastner, "S2n2: A fpga accelerator for streaming spiking neural networks," in *The 2021 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays*, 2021, pp. 194–205.
- [11] R. Karakchi and J. D. Bakos, "Napoly: A non-deterministic automata processor overlay," *ACM Transactions on Reconfigurable Technology and Systems*, vol. 16, no. 3, pp. 1–25, 2023.
- [12] H. Fang, Z. Mei, A. Shrestha, Z. Zhao, Y. Li, and Q. Qiu, "Encoding, model, and architecture: Systematic optimization for spiking neural network in fpgas," in *Proceedings of the 39th International Conference on Computer-Aided Design*, 2020, pp. 1–9.
- [13] J. Smith, "A roadmap for reverse-architecting the brain's neocortex," 2019.
- [14] Q. Chen, C. Gao, and Y. Fu, "Cerebron: a reconfigurable architecture for spatiotemporal sparse spiking neural networks," vol. 30, no. 10. IEEE, 2022, pp. 1425–1437.
- [15] K. Cheung, S. R. Schultz, and W. Luk, "A large-scale spiking neural network accelerator for fpga systems," in *Artificial Neural Networks and Machine Learning–ICANN 2012: 22nd International Conference on Artificial Neural Networks, Lausanne, Switzerland, September 11–14, 2012, Proceedings, Part I* 22. Springer, 2012, pp. 113–120.
- [16] Z. Li, E. Lemaire, N. Abderrahmane, S. Bilavarn, and B. Miramond, "Efficiency analysis of artificial vs. spiking neural networks on fpgas," *Journal of Systems Architecture*, vol. 133, p. 102765, 2022.
- [17] J. Li, G. Shen, D. Zhao, Q. Zhang, and Z. Yi, "Firefly: A high-throughput and reconfigurable hardware accelerator for spiking neural networks," *arXiv preprint arXiv:2301.01905*, 2023.
- [18] H. Li, X. Fan, L. Jiao, W. Cao, X. Zhou, and L. Wang, "A high performance fpga-based accelerator for large-scale convolutional neural networks," in *2016 26th International Conference on Field Programmable Logic and Applications (FPL)*. IEEE, 2016, pp. 1–9.
- [19] J. Zhang and J. Li, "Improving the performance of opencl-based fpga accelerator for convolutional neural network," in *Proceedings of the 2017 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays*, 2017, pp. 25–34.
- [20] H. Tian, Y. Ibrahim, R. Chen, C. Jin, S. Shi, J. Xing, J. Li, and L. Chen, "Evaluation of seu impact on convolutional neural networks based on bram and cram in fpgas," *Microelectronics Reliability*, vol. 144, p. 114974, 2023.
- [21] S. Mittal, "A survey of fpga-based accelerators for convolutional neural networks," *Neural computing and applications*, vol. 32, no. 4, pp. 1109–1139, 2020.
- [22] A. Podili, C. Zhang, and V. Prasanna, "Fast and efficient implementation of convolutional neural networks on fpga," in *2017 IEEE 28th international conference on application-specific systems, architectures and processors (ASAP)*. IEEE, 2017, pp. 11–18.
- [23] W. Zhao, H. Fu, W. Luk, T. Yu, S. Wang, B. Feng, Y. Ma, and G. Yang, "F-cnn: An fpga-based framework for training convolutional neural networks," in *2016 IEEE 27th international conference on application-specific systems, architectures and processors (ASAP)*. IEEE, 2016, pp. 107–114.
- [24] R. Karakchi and N. Robertson, "An approach to mitigate cnn complexity on domain-specific architectures," in *Proceedings of the Second International Conference on Advances in Computing Research (ACR'24)*, ser. Lecture Notes in Networks and Systems, vol. 956, 2024.
- [25] Y. Li, Z. Liu, K. Xu, H. Yu, and F. Ren, "A gpu-outperforming fpga accelerator architecture for binary convolutional neural networks," *ACM Journal on Emerging Technologies in Computing Systems (JETC)*, vol. 14, no. 2, pp. 1–16, 2018.
- [26] S. Zhao, S. Gao, R. Wang, Y. Wang, F. Zhou, and N. Guo, "Acceleration and implementation of convolutional neural networks based on fpga," *Digital Signal Processing*, vol. 141, p. 104188, 2023.
- [27] J. Wu, Y. Wang, L. Lu, C. Chen, and Z. Li, "A high-speed and low-power fpga implementation of spiking convolutional neural network using logarithmic quantization," in *2023 19th International Conference on Natural Computation, Fuzzy Systems and Knowledge Discovery (ICNC-FSKD)*. IEEE, 2023, pp. 1–8.
- [28] R. Li, B. Jiang, and H. Xu, "Mixed dsp packing method for convolutional neural network on fpga," in *Sixth International Conference on Computer Information Science and Application Technology (CISAT 2023)*, vol. 12800. SPIE, 2023, pp. 578–585.
- [29] Y. Chen and M. S. Abdelfattah, "Bramac: Compute-in-bram architectures for multiply-accumulate on fpgas," *arXiv preprint arXiv:2304.03974*, 2023.
- [30] A. Carpegna, A. Savino, and S. Di Carlo, "Spiker: an fpga-optimized hardware accelerator for spiking neural networks," in *2022 IEEE Computer Society Annual Symposium on VLSI (ISVLSI)*. IEEE, 2022, pp. 14–19.
- [31] Y. LeCun, Y. Bengio, and G. Hinton, "Deep learning," *nature*, vol. 521, no. 7553, pp. 436–444, 2015.
- [32] D. F. Khatiboun, Y. Rezaeiyan, M. Ronchini, M. Sadeghi, M. Zamani, and F. Moradi, "Digital hardware implementation of resume learning algorithm for spiking neural networks," in *2023 45th Annual International Conference of the IEEE Engineering in Medicine & Biology Society (EMBC)*. IEEE, 2023, pp. 1–4.
- [33] G. Leone, L. Raffo, and P. Meloni, "On-fpga spiking neural networks for end-to-end neural decoding," *IEEE Access*, 2023.
- [34] M. Ahmad, L. Zhang, K. T. W. Ng, and M. E. Chowdhury, "Complex-exponential-based bio-inspired neuron model implementation in fpga using xilinx system generator and vivado design suite," *Biomimetics*, vol. 8, no. 8, p. 621, 2023.
- [35] Y. Umuroglu, N. J. Fraser, G. Gambardella, M. Blott, P. Leong, M. Jahre, and K. Visser, "Finn: A framework for fast, scalable binarized neural network inference," in *Proceedings of the 2017 ACM/SIGDA international symposium on field-programmable gate arrays*, 2017, pp. 65–74.
- [36] R. Karakchi and J. Frierson, "Towards a scalable spiking neural network," in *Proceedings of the Second International Conference on Advances in Computing Research (ACR'24)*, ser. Lecture Notes in Networks and Systems, vol. 956, 2024.
- [37] E. Garzón, L. Yavits, A. Teman, and M. Lanuzza, "Approximate content-addressable memories: a review," *Chips*, vol. 2, no. 2, pp. 70–82, 2023.
- [38] S. Panchapakesan, Z. Fang, and J. Li, "Syncnn: Evaluating and accelerating spiking neural networks on fpgas," *ACM Transactions on Reconfigurable Technology and Systems*, vol. 15, no. 4, pp. 1–27, 2022.
- [39] M. Gungor, K. Huang, S. Ioannidis, and M. Leeser, "Optimizing designs using several types of memories on modern fpgas," in *2022 IEEE High Performance Extreme Computing Conference (HPEC)*. IEEE, 2022, pp. 1–7.
- [40] T. Gao, B. Deng, J. Wang, and G. Yi, "Highly efficient neuromorphic learning system of spiking neural network with multi-compartment leaky integrate-and-fire neurons," *Frontiers in Neuroscience*, vol. 16, p. 929644, 2022.
- [41] I. Xilinx, "Zynq ultrascale+ mpsoc zcu104 evaluation kit," <https://www.xilinx.com/products/boards-and-kits/zcu104.html>, 2019.